Applicable Algebra in Engineering, Communication and Computing
https://doi.org/10.1007/s00200-025-00711-8

ORIGINAL PAPER



Classification of quadratic APN functions with coefficients in $\mathbb{F}_2$ in dimension 10

Yuyin Yu[1] · Jingchen Li[1] · Nadiia Ichanska[2] · Nikolay Kaleyski[2]

Received: 15 January 2025 / Revised: 22 October 2025 / Accepted: 24 October 2025
© The Author(s) 2025

Abstract
Yu et al. described an algorithm for conducting computational searches for quadratic APN functions over the finite field $\mathbb{F}_{2^n}$, and used this algorithm to give a classification of all quadratic APN functions with coefficients in $\mathbb{F}_2$ for dimensions n up to 9. In this paper, we speed up the running time of that algorithm by a factor of approximately $\frac{2^n}{n^2}$. Based on this result, we give a complete classification of all quadratic APN functions over $\mathbb{F}_{2^{10}}$ with coefficients in $\mathbb{F}_2$. We also perform some partial computations for quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^{11}}$, and conjecture that they form 6 CCZ-inequivalent classes which also correspond to known APN functions.

Keywords Boolean functions · Almost perfect nonlinear · Ortho-derivative · Quadratic functions

✉ Nikolay Kaleyski
Nikolay.Kaleyski@uib.no

Yuyin Yu
yuyuyin@163.com

Jingchen Li
lijingchen0702@qq.com

Nadiia Ichanska
Nadiia.Ichanska@uib.no

[1] College of Mathematics and Information Science, Guangzhou University, Guangzhou, China

[2] Department of Informatics, University of Bergen, Bergen, Norway

Published online: 10 December 2025

1 Introduction

Let n be a positive integer. We denote by $\mathbb{F}_{2^n}$ the finite field with 2^n elements, and by $\mathbb{F}_{2^n}^\star$ its multiplicative group; we will refer to n as the **dimension** of the finite field. A mapping $F : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ is called an **(n , n)-function**. An (n, n)-function F is said to have **differential uniformity** $\delta(F)$ if

$$\delta(F) = \max_{a \in \mathbb{F}_{2^n}^\star, b \in \mathbb{F}_{2^n}} \#\Delta_F(a, b),$$

where $\Delta_F(a, b) = \{x \in \mathbb{F}_{2^n} : F(x + a) + F(x) = b\}$, and $\#S$ is the cardinality of a set S. If $\delta(F) \leq d$ for some positive integer d, then we say that F is **differentially d-uniform**. From the point of view of cryptography, where (n, n)-functions are frequently used as so-called *substitution boxes*, or *S-boxes*, in the design of symmetric block ciphers, it is desirable for $\delta(F)$ to be as low as possible since this indicates a better resistance to differential cryptanalysis. It is easy to see that $\delta(F)$ is always even and can not be equal to zero, and therefore the lowest value of $\delta(F)$ is 2. If $\delta(F) = 2$, then F is called **almost perfect nonlinear (APN)**. Consequently, APN functions provide the best possible resistance to differential attacks, and have attracted significant attention from researchers for this reason. However, as a "side effect" of their cryptographic strength, APN functions tend to lack obvious structure or properties that can be used to construct or characterize them. Finding APN functions is thus not an easy task, and it becomes even more challenging to find functions that combine APN-ness with other desirable properties. Ideally, one would have a classification of all APN functions for a given dimension n, but this is an extremely challenging task, even if only functions from some particular subclass are considered. The vast majority of known APN functions are quadratic, and most of the literature on APN functions concerns primarily quadratic APN functions. Considering quadratic functions all of whose coefficients belong to $\mathbb{F}_2$ further simplifies the classification problem, although it does remain very hard. The last results in this direction are from 2020 [27] where a classification of quadratic APN functions over $\mathbb{F}_{2^n}$ with coefficients in $\mathbb{F}_2$ is performed for $n \leq 9$, and it is shown that applying the approach described in that work to dimensions beyond 9 is infeasible. In this paper, we improve the method from [27], which significantly speeds up the computation and allows us to obtain a complete classification for $n = 10$ and some partial results for $n = 11$; based on these, we also conjecture the exact form of the classification for $n = 11$.

We now give a brief survey of previous classifications and searches for APN functions. Edel et al. [18] found over $\mathbb{F}_{2^{10}}$ the first quadratic APN function that is CCZ-inequivalent [16] to any power function. Brinkmann and Leander [5] showed that when $n < 6$, all APN functions on $\mathbb{F}_{2^n}$ are CCZ-equivalent to power functions, and the APN functions listed in [19] already contain all APN power functions for $n = 4$ and $n = 5$. Dillon et al. [6] first listed 13 CCZ-inequivalent quadratic APN functions on $\mathbb{F}_{2^6}$, while Edel [17] proved that Dillon's list in fact includes all CCZ-inequivalent classes of quadratic APN functions over $\mathbb{F}_{2^6}$.

When $n = 7$, Dillon et al. [6] listed 15 CCZ-inequivalent quadratic APN functions on $\mathbb{F}_{2^7}$, while Edel and Pott [19] expanded this number to 16. Yu et al. [29] found

another 471 new CCZ-inequivalent classes. Kalgin and Idrisova [20] found another new APN function and showed that all quadratic APN functions on $\mathbb{F}_{2^7}$ are included in these 488 CCZ-inequivalent classes.

When $n = 8$, Dillon et al. [6] listed 11 CCZ-inequivalent quadratic APN functions on $\mathbb{F}_{2^8}$, while Edel and Pott [19] expanded this number to 22. Yu et al. [28, 29] found 8157 new CCZ-inequivalent classes by using a computational search based on a matrix representation of quadratic functions[1]. Weng et al. [24] found 10 new ones. Taniguchi [22] found 2 new ones, and Budaghyan et al. [7] found yet another new one. Further, Beierle et al. [1, 2] found 12,921 new CCZ-inequivalent APN functions using a computational approach based on so-called self-equivalences. Yu and Perrin [26] found another 5412 new APN functions and conjectured that the total number would exceed 50000. Consequently, Beierle et al. [4] found 6368 new APN functions. Up to now, there are 32,893 known classes of CCZ-inequivalent quadratic APN functions over $\mathbb{F}_{2^8}$.

Yu et al. [27] described an improvement of the algorithm from [29] in the case when the functions have coefficients in $\mathbb{F}_2$, and gave a classification of all quadratic APN functions with coefficients in $\mathbb{F}_2$ for dimensions up to 9. Beierle et al. [1, 2] found 35 CCZ-inequivalent quadratic APN functions on $\mathbb{F}_{2^9}$ using the self-equivalence approach.

By the year 2022, a total of 20 CCZ-inequivalent quadratic APN instances over $\mathbb{F}_{2^{10}}$ have been identified. Besides the functions equivalent to monomials, 15 quadratic CCZ-inequivalent APN instances over $\mathbb{F}_{2^{10}}$ are given by [10–12, 21], and [22]. Recent progress has been achieved in [2], in which 5 new APN instances were found in dimension 10 (the look-up tables of these 5 APN instances are available in [3]). Shortly thereafter, Lijing Z. et al. [30] reported 4 CCZ- inequivalent APN instances over $\mathbb{F}_{2^{10}}$. Budaghyan et al. [8] obtained 6 previously unknown classes of APN functions by adding 5, 6, or 7 terms with coefficients in $\mathbb{F}_{2^2}$ to x^3, x^9, or x^{33}. They verified that none of these functions are CCZ-equivalent to a permutation by using SboxU [23].

We note that despite the obvious power of many of these methods, it is exceptionally difficult to perform exhaustive searches, and to provide complete classification of the functions of interest, even for relatively low values of n and for specific subclasses of functions. To summarize the above overview, the only full classification of APN functions in general is for $n < 5$ [5]; quadratic APN functions have only been classified for $n = 6$ [17] and $n = 7$ [20]; and quadratic APN functions with coefficients in $\mathbb{F}_2$ have been classified up to $n = 9$ [27]. In this paper, we provide a full classification of quadratic APN functions with coefficients in $\mathbb{F}_2$ for $n = 10$.

Many scholars have also studied how to construct infinite families of quad-ratic APN polynomials over $\mathbb{F}_{2^n}$. Readers can refer to [15] for a comprehensive summary on the state of knowledge of APN functions.

In this paper, we concentrate on quadratic homogeneous functions

[1] The *eprint* version [28] was updated with additional computational results after the publication of the journal version [29], hence why we cite both versions.

$$F(x) = \sum_{0 \le i < j \le n-1} c_{i,j} x^{2^i + 2^j},$$

over $\mathbb{F}_{2^n}$ where $c_{i,j} \in \mathbb{F}_2$, and we investigate when they are APN. We speed up the algorithm in [27] by a factor of approximately $\frac{2^n}{n^2}$ times. Based on this result, we were able to conduct an exhaustive search in dimension 10 (previously, only a partial search had been possible). After running 22 Magma [31] scripts in parallel for about 100 days on $\mathbb{F}_{2^{10}}$, we have obtained 29,088 quadratic homogeneous APN functions with coefficients in $\mathbb{F}_2$. Using the differential spectra of their ortho-derivatives [13], these APN functions can be divided into 3 CCZ-inequivalent classes corresponding to the functions x^3, x^9 and $x^3 + \mathrm{Tr}(x^9)$, where the trace function $\mathrm{Tr} : \mathbb{F}_{2^n} \to \mathbb{F}_2$ is defined by $\mathrm{Tr}(x) = \sum_{i=0}^{n-1} x^{2^i}$ for $x \in \mathbb{F}_{2^n}$. Using the equivalence algorithm from [9], we show that any two of these functions having the same differential spectrum of the ortho-derivative are CCZ-equivalent to each other. Thus, all quadratic APN functions over $\mathbb{F}_{2^{10}}$ with coefficients in $\mathbb{F}_2$ fall into these three CCZ-equivalence classes. However, we found a shorter representative $x^{513} + x^{192} + x^{96} + x^{48} + x^{33} + x^{18} + x^{12}$ for $x^3 + \mathrm{Tr}(x^9)$. This relates to Problem 1 proposed in [27]: given an (n, n)-function F, find a function G that is CCZ-equivalent to F and has the shortest polynomial representation. Based on this, we can confirm that any quadratic APN function over $\mathbb{F}_{2^n}$ with coefficients in $\mathbb{F}_2$ is CCZ-equivalent to a function having no more than n terms with non-zero coefficients for all dimensions n up to 10.

When $n = 11$, we estimate that a complete search using the same method would take at least 70 years on a single CPU core, and thus a complete classification does not appear to be feasible. However, we perform a partial search and find 6 CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$. These functions are all CCZ-equivalent to known ones. Based on the structure of the search results that we observe for $n = 10$, we conjecture that these are the only 6 CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^{11}}$.

2 Preliminaries

We introduce some notions and results which will be useful in the sequel.

We say that an (n, n)-function A is **affine** if for any $x, y, z \in \mathbb{F}_{2^n}$ we have $A(x) + A(y) + A(z) = A(x + y + z)$. If an affine function satisfies $A(0) = 0$, then we say that it is **linear**.

Definition 1 Let F and F' be two functions from $\mathbb{F}_{2^n}$ to $\mathbb{F}_{2^n}$. We say that F and F' are **EA-equivalent** (Extended affine equivalent) if we can write F' as

$$F'(x) = A_1(F(A_2(x))) + A_3(x),$$

where A_1 and A_2 are affine permutations of $\mathbb{F}_{2^n}$, and A_3 is an affine function over $\mathbb{F}_{2^n}$.

We say that F and F' are **CCZ-equivalent** (Carlet-Charpin-Zinoviev equivalent) [16], if there exists an affine permutation of $\mathbb{F}_{2^n} \times \mathbb{F}_{2^n}$, or equivalently $\mathbb{F}_{2^{2n}}$, which maps G_F onto $G_{F'}$, where $G_F = \{(x, F(x)) : x \in \mathbb{F}_{2^n}\}$ is the graph of F.

Definition 2 We denote by $\mathcal{F}_n$ a maximal subset of $\mathbb{F}_{2^n}^*$ such that $x \in \mathcal{F}_n$ implies $x^{2^k} \notin \mathcal{F}_n$ for any $1 \leq k \leq n-1$.

Later in the paper, we will introduce an equivalence relation which we call "cyclic equivalence" between the elements of $\mathbb{F}_{2^n}^*$. A set $\mathcal{F}_n$ will then be a set of representatives from the equivalence classes induced by this relation on $\mathbb{F}_{2^n}^*$.

Definition 3 [29] Let $B \in \mathbb{F}_{2^n}^m$ be a vector $B = (\eta_0, \eta_1, \ldots, \eta_{m-1})$ where $\eta_i \in \mathbb{F}_{2^n}$ for $0 \leq i \leq m-1$. Then $\mathrm{Span}(B) = \mathrm{Span}(\eta_0, \eta_1, \ldots, \eta_{m-1})$ is the subspace spanned by $\{\eta_0, \eta_1, \ldots, \eta_{m-1}\}$ over $\mathbb{F}_2$. The dimension of this subspace is denoted by $\mathrm{Rank}(B) = \mathrm{Rank}(\eta_0, \eta_1, \ldots, \eta_{m-1})$, and is referred to as the rank of B over $\mathbb{F}_2$.

Suppose $\{\alpha_0, \alpha_1, \ldots, \alpha_{n-1}\}$ is a basis of $\mathbb{F}_{2^n}$ over $\mathbb{F}_2$, and $\eta_i = \sum_{j=0}^{n-1} \lambda_{i,j}\alpha_j$ for $0 \leq i \leq m-1$, with $\lambda_{i,j} \in \mathbb{F}_2$ for $0 \leq i, j \leq n-1$. We define an m-by-n matrix $\Lambda \in \mathbb{F}_2^{m \times n}$ by $\Lambda[i, j] = \lambda_{i,j}$, so that each row of Λ contains the coordinates of η_i with respect to the basis. Then the rank of B is equal to the rank of Λ.
While the above is true for any basis, we assume throughout the rest of the paper that $\{\alpha_0, \alpha_1, \ldots, \alpha_{n-1}\}$ is a normal basis, such that $\alpha_{i+1} = \alpha_i^2$ for $0 \leq i \leq n-2$, and $M_\alpha \in \mathbb{F}_{2^n}^{n \times n}$ is defined by $M_\alpha[i, u] = \alpha_u^{2^i}$ for $0 \leq u, i \leq n-1$. This allows us to exploit the diagonal properties of QAMs described below, and simplifies some computations.

Definition 4 [29] Let $H \in \mathbb{F}_{2^n}^{m \times k}$ with $1 \leq m, k \leq n$. We say that H is **proper** if every nonzero $\mathbb{F}_2$-linear combination of the m rows of H has rank at least $k-1$.

Definition 5 [29] Let H be an $n \times n$ matrix defined on $\mathbb{F}_{2^n}$. Then H is called a **QAM** (quadratic APN matrix) if:

(i) H is symmetric and the elements in its main diagonal are all zeros;
(ii) H is proper, i.e. every nonzero linear combination of the n rows (or, equivalently, columns, due to H being symmetric) of H has rank $n-1$.

Theorem 1 *[27] Let* $F(x) = \sum_{0 \leq t < i \leq n-1} c_{i,t} x^{2^i + 2^t}$ *be a quadratic homogeneous* (n, n)*-function. Define an* $n \times n$ *matrix* C_F *by* $C_F[t, i] = C_F[i, t] = c_{i,t}$ *for* $0 \leq t < i \leq n-1$ *and* $C_F[i, i] = 0$ *for* $0 \leq i \leq n-1$*. Finally, take*

$$H = M_\alpha^t C_F M_\alpha.$$

Then

$$H[u+1, v+1] = H[u, v]^2 \tag{1}$$

(with the indices taken modulo n) for $0 \leq u, v \leq n-1$ *if and only if* $c_{i,t} \in \mathbb{F}_2$ *for* $0 \leq t < i \leq n-1$.

Intuitively, when the coefficients of the function all belong to $\mathbb{F}_2$, the entries of H have the property that, along each diagonal, each subsequent value is simply the square of the previous one. Thus, knowing a single value of H gives us knowledge of the values on its entire diagonal.

Proposition 1 *[27] Suppose* $F_1 \in \mathbb{F}_{2^n}[x]$ *is a homogeneous quadratic APN function with coefficients in* $\mathbb{F}_2$*, and H is its corresponding QAM. Let* H' *be the matrix defined by* $H'[i,j] = H[i,j]^2$ *for* $0 \leq i, j < n$*. Then* H' *is also a QAM, and its corresponding function* $F_2 \in \mathbb{F}_2[x]$ *is EA-equivalent to* F_1.

According to [25], two quadratic APN functions are CCZ-equivalent if and only if they are EA-equivalent. In order to categorize our functions into distinct EA-equivalence classes (and thus distinct CCZ-classes), we computed the values of the differential spectra of the ortho-derivatives [13] for each function, and then applied the algorithm from [9] to every pair of functions having the same ortho-derivative differential spectrum. First, let us recall the definition of the ortho-derivative.

Definition 6 [13] Let $F : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ be a quadratic APN function, and let $x \cdot y$ denote a scalar product, that is, a symmetric bivariate function on $\mathbb{F}_{2^n}$ such that $x \mapsto a \cdot x$ is a non-zero linear form for any $a \in \mathbb{F}_{2^n}^*$. Then the *ortho-derivative* of F is the unique function $\pi_F : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ such that $\pi_F(0) = 0$, $\pi_F(a) \neq 0$ if $a \neq 0$, and such that

$$\pi_F(a) \cdot \big(F(x+a) + F(x) + F(a) + F(0)\big) = 0 \tag{2}$$

for all $a \in \mathbb{F}_{2^n}^*$ and all $x \in \mathbb{F}_{2^n}$.
According to [13], if two quadratic APN functions are EA-equivalent, then their ortho-derivatives are affine equivalent, which implies that their ortho-derivatives must have the same extended Walsh spectra and differential spectra. Thus, if the ortho-derivatives of two quadratic APN functions have different extended Walsh spectra or differential spectra, then they are necessarily EA-inequivalent. It is known that the Walsh and differential spectra of the ortho-derivatives are very efficient for distinguishing between CCZ-inequivalent quadratic APN functions. Since they can be computed very fast (for $n \leq 11$, it is efficient enough to find the ortho-derivative of a function by, e.g., brute-force, i.e. guessing the value of the orthoderivative at every non-zero input $a \in \mathbb{F}_{2^n}^*$ and verifying whether this really satisfies the definition), performing this before applying an equivalence algorithm can significantly speed up the process of categorizing a set of functions into CCZ-equivalence classes.

The algorithm described in [9] allows one to efficiently test whether two given functions $F, G : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ that are 3-to-1 on $\mathbb{F}_{2^n}^*$ are linearly equivalent, i.e.,

whether there exist linear permutations L_1 and L_2 of $\mathbb{F}_{2^n}$ such that $L_1 \circ F \circ L_2 = G$. The method is a variation of the one of Biryukov et al. for testing linear equivalence between permutations [14], which exploits the structural properties of 3-to-1 APN functions for more efficient computations. The basic idea is to guess the values of L_2 on some elements of a basis of $\mathbb{F}_{2^n}$ (viewed as a vector space over $\mathbb{F}_2$), which then allows values of L_1 to be deduced, and vice versa (in the case when F and G are permutations, this is straightforward, whereas for 3-to-1 functions one needs to consider the structure of the 3-to-1 functions when deducing values of L_2 from those of L_1). While the primary method from [9] handles only linear equivalence, in practice two quadratic 3-to-1 APN functions that are CCZ-equivalent are also linearly equivalent. This empirical observation turned out to be true for the functions that we found as well, and thus using this algorithm was sufficient to confirm equivalence (since linear equivalence of two functions implies their CCZ-equivalence) and to complete the classification. An implementation of this algorithm (by one of the original authors) can be found at https://github.com/telegrin/3to1_vbf_equivalence.

3 A faster method to determine QAM

According to the definition of QAM (See Definition 5), if an $n \times n$ matrix H is a QAM, then every nonzero linear combination of the n rows of H has rank $n-1$. Note that there are $2^n - 1$ nonzero linear combinations of the n rows of H. In this section, we will investigate how to reduce the data to be tested for determining quadratic APN functions with coefficients in $\mathbb{F}_2$.

3.1 Reducing the data to be tested

We will prove that about $\frac{2^n}{n}$ nonzero linear combinations of the n rows of H need to be tested in order to determine whether a matrix corresponds to a quadratic APN function with coefficients in $\mathbb{F}_2$. Intuitively, we already know that "going down diagonals" in H results in all entries being squared. Consequently, the values in two adjacent rows are simply the squares of each other, which implies that any two adjacent rows have the same rank.

Theorem 2 *Let $H \in \mathbb{F}_{2^n}^{n\times n}$, and $H[u,u] = 0$, $H[u,v] = H[v,u]$, $H[u+1, v+1] = H[u,v]^2$ (with the indices taken modulo n) for $0 \le v, u \le n-1$. Then*

$$Rank\left(\sum_{u=0}^{n-1} \lambda_u H[u,:]\right) = Rank\left(\sum_{u=0}^{n-2} \lambda_u H[u+1,:] + \lambda_{n-1} H[0,:]\right), \quad (3)$$

where $(\lambda_0, \lambda_1, \ldots, \lambda_{n-1}) \in \mathbb{F}_2^n \backslash \{0\}$, and H[u, :] is the u-th row of H.

Proof According to the properties of H, we have

$$Span\left(\sum_{u=0}^{n-2}\lambda_u H[u+1,:]+\lambda_{n-1}H[0,:]\right)=Span\left(\left(\sum_{u=0}^{n-1}\lambda_u H[u,:]\right)^2\right)$$

which implies that

$$\begin{aligned}Rank\left(\sum_{u=0}^{n-2}\lambda_u H[u+1,:]+\lambda_{n-1}H[0,:]\right)&=Rank\left(\sum_{u=0}^{n-1}\lambda_u H[u,:]\right)^2\\&=Rank\left(\sum_{u=0}^{n-1}\lambda_u H[u,:]\right).\end{aligned}$$

□

Example 1 Let $n=5$, define

$$H=\begin{pmatrix}0 & a & b & b^8 & a^{16}\\ a & 0 & a^2 & b^2 & b^{16}\\ b & a^2 & 0 & a^4 & b^4\\ b^8 & b^2 & a^4 & 0 & a^8\\ a^{16} & b^{16} & b^4 & a^8 & 0\end{pmatrix}\in\mathbb{F}_{2^5}^{5\times 5}.$$

(i) Let $(\lambda_0,\lambda_1,\ldots,\lambda_{n-1})=(1,0,0,0,0)$. Then

$$H[0,:]^2=(0,a^2,b^2,b^{16},a)\text{ and }H[1,:]=(a,0,a^2,b^2,b^{16}),$$

which implies that

$$Span(H[0,:]^2)=Span(H[1,:]).$$

Therefore, we have

$$Rank(H[1,:])=Rank(H[0,:]^2)=Rank(H[0,:]).$$

Similarly, we have

$$Rank(H[1,:])=Rank(H[2,:])=Rank(H[3,:])=Rank(H[4,:]).$$

(ii) Let $(\lambda_0,\lambda_1,\ldots,\lambda_{n-1})=(1,1,0,0,0)$, Similarly as in (i), we have

$$Span((H[0,:]+H[1,:])^2)=Span(H[1,:]+H[2,:]).$$

Therefore,

$$\begin{aligned} Rank(H[0,:]+H[1,:]) =& Rank(H[1,:]+H[2,:]) \\ =& Rank(H[2,:]+H[3,:]) \\ =& Rank(H[3,:]+H[4,:]) \\ =& Rank(H[4,:]+H[1,:]). \end{aligned}$$

Based on Theorem 2 and Example 1, we can conclude that only about $\frac{2^n}{n}$ nonzero linear combinations need to be tested. We will give an exact number in the following.

Two elements $a=(a_0,a_1,\ldots,a_{n-1})$, $b=(b_0,b_1,\ldots,b_{n-1})\in\mathbb{F}_2^n$ are called **cyclically equivalent** if $a=(b_k,b_{k+1},\ldots,b_{k+n-1})$ (with the indices taken modulo n) for some integer *k*. We will refer to this equivalence relation as **cyclic equivalence**. Let S_t for $t=0,1,\ldots,K$, for some positive integer *K*, be all equivalence classes of $\mathbb{F}_{2^n}$ under cyclic equivalence, so that $S_0,S_1,\ldots,S_K$ form the partition (quotient set) of $\mathbb{F}_{2^n}$ induced by cyclic equivalence. According to Theorem 2, we can get the following corollary.

Corollary 1 *Let $S_0=\{0\}$, and $S_0,S_1,\ldots,S_K$ be the quotient set induced by cyclic equivalence. Suppose H is the same as in Theorem 2, then K nonzero linear combinations of the n rows of H are enough to determine whether it is a QAM.*

Proof In fact, only one element from each S_t $(1\le t\le K)$ needs to be considered in order to test the APN property of *H*. According to Theorem 2, for any cyclically equivalent elements $a=(a_0,a_1,\ldots,a_{n-1}), b=(b_0,b_1,\ldots,b_{n-1})\in S_i$, we must have

$$\mathrm{Rank}\left(\sum_{u=0}^{n-1}a_uH[u,:]\right)=\mathrm{Rank}\left(\sum_{u=0}^{n-1}b_uH[u,:]\right).$$

□

Note that $K\approx\frac{2^n}{n}$. Thus, by using Theorem 2, we can reduce the amount of data that needs to be computed to $\frac{1}{n}$ of the original, thereby achieving approximately an *n*-fold speedup. However, the exact speed up heavily depends on how we implement the algorithm. For example, suppose we must compute $\mathrm{Rank}(H[0,:]+H[1,:])$, $\mathrm{Rank}(H[0,:]+H[1,:]+H[2,:])$ and $\mathrm{Rank}(H[0,:]+H[1,:]+H[2,:]+H[3,:])$, then $H[0,:]+H[1,:]$ would be calculated 3 times. Avoiding such redundant computations is a key factor in speeding up the runtime.

3.2 Gaussian elimination

The most time-consuming part of the algorithm in [27] is testing the QAM property of a given matrix. According to Definitions 3 and 5, there are two ways to speed up this algorithm:

(i) by reducing the amount of data that needs to be tested, as discussed in Sect. 3.1;
(ii) by speeding up the computation of the rank, which we address in the following.

Based on Definition 3, if $B = (\eta_0, \eta_1, \ldots \eta_{m-1}) \in \mathbb{F}_{2^n}^m$, then $\mathrm{Rank}(B)$ is the dimension of $\mathrm{Span}(B)$. Therefore, the time complexity of computing $\mathrm{Rank}(B)$ is $O(2^n)$ when generating $\mathrm{Span}(B)$ directly. However, if we expand B as an $m \times n$ matrix $\Lambda \in \mathbb{F}_2^{m \times n}$, and then calculate the rank of Λ, the time complexity becomes $O(nm^2)$. Since this paper considers only the case of $m = n$, this method can result in approximately $\frac{2^n}{n^3}$ times acceleration. Note that $\frac{2^n}{n^3} > 1$ holds only when $n \geq 10$; therefore, the acceleration effect becomes significant for larger n. Moreover, the larger the value of n, the greater the factor $\frac{2^n}{n^3}$, so the speed-up grows rapidly with increasing dimensions.

Based on the results of Sect. 3.1 and 3.2, an overall acceleration factor of $n * \frac{2^n}{n^3} = \frac{2^n}{n^2}$ can be achieved.

3.3 Algorithm to determine QAM

Based on Sects. 3.1 and 3.2, we present a faster algorithm for enumerating all QAMs. The core of the procedure is the function $\mathrm{IsQAM}()$, provided in pseudocode in Algorithm 1, together with several auxiliary functions necessary for its implementation. The implementation leverages the computational improvement described in Sects. 3.1 and 3.2.

The $\mathrm{IsQAM(H)}$ function takes a matrix H and determines whether it is a QAM by computing the ranks of linear combinations of its rows. As discussed in Sect. 3.1, only certain linear combinations need to be considered, which reduces the computation time. These combinations are stored in the array RC (short for "row combinations"). This allows us to avoid redundant additions by reusing linear combinations that have already been computed. For example, if one linear combination that we need to test is $H[1,:] + H[2,:]$ and another is $H[1,:] + H[2,:] + H[4,:]$, then the latter can be obtained from the former (which is stored in RC) by simply adding the fourth row of H, without recomputing the sum $H[1,:] + H[2,:]$.

We need to determine which rows of H must be summed with which entries of RC in order to obtain all linear combinations that have to be checked. To do this, we first compute a list of representatives $\mathcal{F}_n$ of the elements of $\mathbb{F}_{2^n}^*$ with respect to cyclic equivalence. This is done using the function $\mathrm{getCyclicEquivalenceRepresentatives}()$ and is stored in the array *CR* (short for "cyclic representatives"), arranged in ascending order. We note that every element $(a_0, a_1, \ldots, a_{n-1})$ can be identified with the integer $\sum_{i=0}^{n-1} a_i \times 2^i \in \mathbb{Z}_{2^n}$. The sets S_i forming the quotient set of $\mathbb{F}_{2^n}^*$ under cyclic equivalence can therefore be seen as sets of integers, and in the algorithm we select the smallest integer within each set as its representative.

The binary expansion of the representatives directly corresponds to the linear combinations of the rows of H that need to be tested. If $\sum_{i=0}^{n-1} a_i \times 2^i$ is one such representative, it corresponds to the linear combination $\sum_{i=0}^{n-1} H[i,:]$. Thus, computing $\mathcal{F}_n$ is useful both for reducing the number of guesses for the first unknown in H

(that is, the first level of the depth-first search) and for verifying the necessary conditions for H to be a QAM.

To avoid redundant additions when computing the linear combinations of the rows, we compute an array of indices $ID[i,j]$ for $i \in \{1,2\}$, $j \in \{1,2,\ldots,K\}$. These indices specify how the linear combinations of the rows of H should be iteratively computed in order to avoid redundant computations. The linear combinations themselves are stored in the array RC, which is initialised at the beginning of the computation to contain only the zero vector, i.e., $RC[1]$ is the zero vector. Subsequent linear combinations are computed as $RC[t+1] = RC[ID[1,t]+1] + H[ID[2,t],:]$ for $t = 1,2,\ldots,K$.

To calculate the index array ID, we observe that if $CR[t] = \sum_{i=1}^{n} a_i \times 2^{i-1}$, then $RC[t+1] = \sum_{i=1}^{n} a_i H[i,:]$. When constructing ID, we ensure that $CR[t] = CR[ID[1,t]] + 2^{ID[2,t]-1}$, which implies $RC[t+1] = RC[ID[1,t]+1] + H[ID[2,t],:]$.

For each linear combination of the rows, we need to check whether it has rank $n-1$. To do so, after computing $RC[t+1]$, we expand it to an $n \times n$ matrix over $\mathbb{F}_2$ and transform it to row-echelon form using Gaussian elimination, from which we obtain the rank of $RC[t+1]$.

In summary, we first select the smallest representative element in each cyclic equivalence set and then establish an iterative relationship between these representative elements. This iterative relationship satisfies the following two properties:

(i) it allows us to test only one representative element in each cyclic equivalence set, which reduces the amount of data to be tested;
(ii) when calculating the sum of any $k+1$ rows of H, only a single row addition is required, which prevents redundant row additions.

For clarity, we give an example illustrating what happens when running $\mathrm{IsQAM(n,H)}$ in Algorithm 1.

Example 2 Let $n = 5$, and let H be the same as in Example 1. We have $RC[1] = (0,0,0,0,0)$, CR=(1, 3, 5, 7, 11, 15, 31), $IDlen = 7$, and

$$ID = \mathrm{GetIndices(n)} = \begin{pmatrix} 0 & 1 & 1 & 2 & 2 & 4 & 6 \\ 1 & 2 & 3 & 3 & 4 & 4 & 5 \end{pmatrix}.$$

Firstly, the array RC stores the nonzero linear combinations of the five rows of H that need to be checked. Based on $RC[t+1] = RC[ID[1,t]+1] + H[ID[2,t],:]$, we have

$$
\begin{aligned}
&RC[2] = RC[1] + H[1,:] = H[1,:],\\
&RC[3] = RC[2] + H[2,:] = H[1,:] + H[2,:],\\
&RC[4] = RC[2] + H[3,:] = H[1,:] + H[3,:],\\
&RC[5] = RC[3] + H[3,:] = H[1,:] + H[2,:] + H[3,:],\\
&RC[6] = RC[3] + H[4,:] = H[1,:] + H[2,:] + H[4,:],\\
&RC[7] = RC[5] + H[4,:] = H[1,:] + H[2,:] + H[3,:] + H[4,:],\\
&RC[8] = RC[7] + H[5,:] = H[1,:] + H[2,:] + H[3,:] + H[4,:] + H[5,:].
\end{aligned}
$$

Note that for any $1 \le t \le IDlen$, only one addition is needed to calculate $RC[t+1]$. Furthermore, in each iteration of the loop, we check whether the Rank of $RC[t+1]$ equals $n-1$.

Next, we discuss the correspondence between $RC[t+1]$ and $CR[t]$ for any $1 \le t \le IDlen$. Note that if $CR[t] = \sum_{i=1}^{5} a_i * 2^{i-1} \mapsto (a_1, a_2, a_3, a_4, a_5)$, then $RC[t+1] = \sum_{i=1}^{n} a_i * H[i,:]$. For example

$$
\begin{aligned}
&CR[1] = 1 \mapsto (1,0,0,0,0) \mapsto RC[2] = H[1,:],\\
&CR[2] = 3 \mapsto (1,1,0,0,0) \mapsto RC[3] = H[1,:] + H[2,:],\\
&CR[3] = 5 \mapsto (1,0,1,0,0) \mapsto RC[4] = H[1,:] + H[3,:],\\
&CR[4] = 7 \mapsto (1,1,1,0,0) \mapsto RC[5] = H[1,:] + H[2,:] + H[3,:],\\
&CR[5] = 11 \mapsto (1,1,0,1,0) \mapsto RC[6] = H[1,:] + H[2,:] + H[4,:],\\
&CR[6] = 15 \mapsto (1,1,1,1,0) \mapsto RC[7] = H[1,:] + H[2,:] + H[3,:] + H[4,:],\\
&CR[7] = 31 \mapsto (1,1,1,1,1) \mapsto RC[8] = H[1,:] + H[2,:] + H[3,:] + H[4,:] + H[5,:].
\end{aligned}
$$

In short, the smallest element from each cyclic equivalence set is stored in *CR* in ascending order. According to Corollary 1, we only need to calculate the rank of the nonzero linear combinations of the five rows of *H* that correspond to *CR*[*t*] for $1 \le t \le 7$. We also note that $S_t \subset \mathbb{F}_2^n$ in Corollary 1. If all elements in S_t are converted to integers, then *CR*[*t*] is the smallest element of this set.

Finally, we explain why we need *ID*=GetIndices(n) in our implementation. The ID stores the subscripts that describe how to iteratively obtain elements with larger indices from those with smaller indices in ID. We first generate *CR* by calling *CR*=getCyclicEquivalenceRepresentatives(n), and then search for the iterative relationship between consecutive elements in *CR* that satisfies $CR[t] = CR[ID[1,t]] + 2^{ID[2,t]-1}$ (for $2 \le t \le 7$). Based on the correspondence between $RC[t+1]$ and *CR*[*t*] ($1 \le t \le 7$) listed above, we can obtain the array ID using $RC[t+1] = RC[ID[1,t]+1] + H[ID[2,t],:]$ for any $1 \le t \le 7$ after adding the initial values of *ID*[1, 1] and *ID*[2, 1]. For example, $CR[4] = 7 = CR[ID[1,4]] + 2^{ID[2,4]-1} = CR[2] + 2^{3-1} = 3 + 4$ implies $RC[5] = RC[ID[1,4]+1] + H[ID[2,4],:] = RC[2] + H[3,:]$.

Input: A positive integer n, and a symmetric matrix $H \in \mathbb{F}_{2^n}^{n\times n}$ with $H[u,u]=0$, and $H[u+1,v+1]=H[u,v]^2$ for $1\le u,v\le n$;
Output: If H is a QAM, return true; otherwise, return false.

1 **function** `IsQAM(n,H)`:
2 ID=`GetIndices(n)`;
3 $IDlen$=`NumberOfColumns(ID)`;
4 $RC=[ZeroVector]$; // $ZeroVector=(0,0,\ldots,0)\in\mathbb{F}_{2^n}^n$
5 **for** $t\in[1..IDlen]$ **do**
6 $rc=RC[ID[1,t]+1]+H[ID[2,t]]$;
7 Expand $rc\in\mathbb{F}_{2^n}^n$ to $MA\in\mathbb{F}_2^{n\times n}$;
8 **if** *GaussRank(MA)* $<n-1$ **then**
9 return false; // Calculate the rank of MA with the Gaussian elimination method
10 **end**
11 Append rc to RC; // $RC[t+1]=RC[ID[1,t]+1]+H[ID[2,t]]$
12 **end**
13 return true;
14 **end function**
15 **function** `GetIndices(n)`: // Return ID with $CR[t]=CR[ID[1,t]]+2^{ID[2,t]-1}$
16 CR=`getCyclicEquivalenceRepresentatives(n)`;
17 $len=\#CR$;
18 Initialize $ID\in\mathbb{Z}^{2\times len}$ with 0;
19 $ID[1,1]=0; ID[2,1]=1$;
20 **for** $t\in[2..len]$ **do**
21 $e=CR[t]$;
22 Select $j\in[1..n]$ with $2^{j-1}<e\le 2^j$;
23 Select $i\in[1..len]$ with $CR[i]==e-2^{j-1}$; // We can always find such i since in Line 32 we select the smallest element in each cyclic equivalent class
24 $ID[1,t]=i; ID[2,t]=j$; // $CR[t]=CR[ID[1,t]]+2^{ID[2,t]-1}$
25 **end**
26 return ID;
27 **end function**
28 **function** `getCyclicEquivalenceRepresentatives(n)`: // Return the smallest element in each cyclic equivalent class
29 CR=[ ];
30 NZ=$[t:t\in[1..2^n-1]]$;
31 **while** *true* **do**
32 $e=NZ[1]$; // Select the smallest element in NZ
33 Append e to the end of CR;
34 $k=e$;
35 **for** $i\in[1..n]$ **do**
36 Delete k from NZ; // Delete any element cyclic equivalent with e
37 $k=2\times k$;
38 **if** $k>2^n$ **then**
39 $k=k-2^n+1$;
40 **end**
41 **end**
42 **if** *NZ is empty* **then**
43 break;
44 **end**
45 **end**
46 return CR;
47 **end function**

Algorithm 1 Determine whether H is a QAM

4 Algorithm and experimental results

In this section, we explain how to generate QAMs with IsQAM(n,H) in Algorithm 1.

4.1 Algorithm

The basic idea of the algorithm is to perform a depth-first search over all possible instantiations of the matrix H, verifying the necessary conditions for being a QAM at every step, and backtracking if they are violated. For the first unknown value in H, we can restrict our guesses to elements from $\mathcal{F}_n$. For subsequent unknowns, we have to consider all possible values from $\mathbb{F}_{2^n}^*$. Recall from the discussion above that knowing one value of H immediately allows us to deduce the values of all other entries on its diagonal. Thanks to this, the matrix has relatively few degrees of freedom, which makes the entire search feasible.

For clarity, we describe the algorithm for the specific case of $n = 5$. The same principle naturally extends to an arbitrary dimension n.

Suppose H is the same as in Example 1. According to Proposition 1, we only need to consider $a \in \mathcal{F}_5$. Then we can get Algorithm 2. We recall that $\mathcal{F}_n$ is defined in Definition 2.

```
1  for a ∈ F_5 do
2      if ( 0  a ) is proper then
           ( a  0 )
3          for b ∈ F*_{2^5} do
                 ( 0   a   b   )
4                if ( a   0   a^2 ) is proper then
                    ( b   a^2 0   )
5                    if IsQAM(5,H) then
6                        Generate APN from H.
7                    end
8                end
9            end
10     end
11 end
```

Algorithm 2 Generate APN functions on $\mathbb{F}_{2^5}$

Algorithm 2 is a special case of the algorithm in [27], except for Line 5. Algorithm 2 can be easily generalized to any n, which we refer to as Algorithm 2 for $n = k$ $(k = 7, 8, 9, 10, 11)$ in the following discussion. According to the results of Sects. 3.1 and 3.2. Algorithm 2 for n is approximately $\frac{2^n}{n^2}$ times faster than the algorithm in [27]. Running Algorithm 2 for $n = 7, 8, 9$ on a computer with an Intel Core i7-7700 CPU at 3.6G GHz takes approximately 14.5 s, 25 min, and 4 days, respectively, to reproduce the results obtained with the algorithm in [27]. When running Algorithm 2 for $n = 10$ on a server equipped with an Intel Xeon 8336C (32 cores/64 threads) CPU at 2.3 GHz, with 22 Magma scripts executed in parallel, the computation takes about 100 days and produces 29088 quadratic homogeneous APN functions with coefficients in $\mathbb{F}_2$. By computing the differential spectra of the ortho-derivatives [13] and then applying the equivalence algorithm from [9], these APN

Table 1 CCZ-inequivalent representatives for all quadratic APN functions with coefficients in $\mathbb{F}_2$ for $n = 10$

x^3
x^9
$x^{513} + x^{192} + x^{96} + x^{48} + x^{33} + x^{18} + x^{12}$

Table 2 Time and APN functions

n	6	7	8	9	10
Time	0.047 s	0.375 s	27.142 s	582.781 s	45 h
Count	4	78	142	26	45

functions can be divided into 3 CCZ-inequivalent classes, all of which correspond to known APN functions. The results are listed in Table 1.

Although no new APN functions over $\mathbb{F}_{2^{10}}$ were found, we identified a shorter representative for $x^3 + \mathrm{Tr}(x^9)$. Therefore, we can conclude that the CCZ-equivalence class of any quadratic APN function over $\mathbb{F}_{2^n}$ for $n \leq 10$ with coefficients in $\mathbb{F}_2$, is already represented by one of the known functions. It was observed in [27] that any quadratic APN function F_1 with coefficients in $\mathbb{F}_2$ appears to be CCZ-equivalent to a quadratic APN function F_2 with at most n nonzero coefficients in $\mathbb{F}_{2^n}$ for $n \leq 9$. We confirm that this conjecture also holds for $n = 10$.

4.2 Further observation

There are only 3 CCZ-inequivalent APN functions among the 29,088 quadratic homogeneous APN functions on $\mathbb{F}_{2^{10}}$, which implies that a huge number of these functions are CCZ-equivalent to each other. Therefore, it is meaningful to exclude some CCZ-equivalent functions in our implementation. Note that in Algorithm 2, $a \in \mathcal{F}_5$ while $b \in \mathbb{F}_{2^5}^*$, and $\#\mathbb{F}_{2^5}^* \approx \#\mathcal{F}_5 \times 5$. Thus, it may be reasonable to restrict $b \in \mathcal{F}_5$ when performing a partial search. The pseudocode for such a partial search is given below as Algorithm 3. As before, we describe the algorithm for the concrete case $n = 11$; however, it naturally generalizes to any dimension n.

In general, if we restrict the range of values for the elements in the left half of the first row to $\mathcal{F}_n$ in Algorithm 2 for a given n, and run it on a computer with an Intel Core $i7 - 7700$ CPU at 3.6 GHz, we obtain the results summarized in Table 2. Column 2 for example indicates that it takes 0.047 seconds to get 4 quadratic homogeneous APN functions over $\mathbb{F}_{2^6}$. These functions contain all CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^n}$ for $n = 6, 7, 8, 9, 10$.

Inspired by the results in Table 2, we now consider the case for $n = 11$. For clarity, we briefly introduce the process of the algorithm. Let $H_{11} \in \mathbb{F}_{2^{11}}^{11\times 11}$. According to the definition of QAM and Theorem 1, if H_{11} is the corresponding QAM of a quadratic APN function with coefficients in $\mathbb{F}_2$, then we must have

$$H_{11}[1,:] = [0, a, b, c, d, e, e^{64}, d^{128}, c^{256}, b^{512}, a^{1024}],$$

for some $a, b, c, d, e \in \mathbb{F}_{2^{11}}$.

Assuming that a, b, c, d, e are unknown variables, we investigate how to assign their values so that H_{11} becomes a QAM. A full search using Algorithm 2 is unfor-

Table 3 CCZ-inequivalent representatives from the partial search for n = 11

x^3
x^5
x^9
x^{17}
x^{33}
$x^3 + \mathrm{Tr}(x^9)$

tunately not feasible; therefore, we perform a partial search by restricting the values of all variables to the set $\mathcal{F}_{11}$. Based on our observations in lower dimensions, it is likely that this partial search will find at least one representative from each CCZ-equivalence class, although confirming that this is indeed a complete classification would require an exhaustive search.

```
for a, b, c, d, e ∈ F_11 do
    if the k-th order submatrix of the upper left corner of H_11 is proper for
     k = 2, 3, 4, 5, 6 then
        if IsQAM(11, H_11) then
            Generate APN from H_11.
        end
    end
end
```

Algorithm 3 Partial search for APN functions on $\mathbb{F}_{2^{11}}$

Algorithm 3 can also be generalized to any n, and we refer to it as Algorithm 3 for n. We have implemented it in Magma, and, based on the discussion above, Table 2 represents the results of running Algorithm 3 for $n = 6, 7, 8, 9,$ and 10.

Algorithm 3 for $n = 11$ is approximately 11^4 times faster than Algorithm 2 for the same dimension. We estimate that, even with these improvements, it would take at least 70 years of running time on a single CPU core to perform a complete search over all functions. Nevertheless, we performed a partial search for $n = 11$, running 15 Magma scripts in parallel on a server equipped with an Intel Xeon 8336C 32C/64T CPU at 2.3 GHz for 12 days, and obtained 29 APN functions. According to the differential and Walsh spectra of their ortho-derivatives, there are at least 6 CCZ-inequivalent classes among these functions, all of which are likely CCZ-equivalent to the known APN classes listed in Table 3. Unfortunately, none of the currently available algorithms for testing CCZ- or EA-equivalence can be applied to these functions to get a complete classification. We leave this as a problem for future work.

Nonetheless, we conjecture that Table 3 includes all CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^{11}}$, since Algorithm 3 for $n = 6, 7, 8, 9,$ and 10 can generate all CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^n}$ for $n = 6, 7, 8, 9,$ and 10.

4.3 Complexity analysis

According to Theorem 1, we only need to check the matrices that satisfy (1), and it is easy to see that there are approximately $2^{n(n-1)/2}$ such matrices. By using Propo-

sition 1, this number can be reduced to $\frac{2^{n(n-1)/2}}{n}$. For each matrix H, we need to run `IsQAM(H)` to check whether the corresponding function is APN. Therefore, the overall complexity is $\frac{2^{n(n-1)/2}}{n}$ times the complexity of running `IsQAM(H)`. According to the definition of QAM, we need to calculate the ranks of all nonzero linear combination of its rows, and their number is $2^n - 1$ (if the matrix is not a QAM, this number may be less than $2^n - 1$; thus, the complexity in this section represents an upper bound). For each row, if a vector space is generated using all elements of this row as generators, the complexity is 2^n. These are the methods used in [27]; therefore, its total complexity is $O(\frac{2^{n(n-1)/2+2n}}{n})$. On the other hand, in Algorithm 2 of this paper, by using the results from Sects. 3.1 and 3.2, the complexity is reduced to $O(n2^{n(n-1)/2+n})$.

In Algorithm 3, we only need to check about $\frac{2^{n(n-1)/2}}{n^{(n-1)/2}}$ matrices which satisfy (1), the remaining parts are the same as in Algorithm 2. Thus, the complexity of Algorithm 3 is $O(\frac{2^{n(n-1)/2+n}}{n^{(n-1)/2}})$.

The space complexity of Algorithms 2 and 3 is the same. Both need to save one $n \times n$ matrix (in $\mathbb{F}_{2^n}^{n\times n}$), and $\frac{2^n}{n}$ rows, where each row (in $\mathbb{F}_{2^n}^{1\times n}$) is expanded into an $n \times n$ matrix (in $\mathbb{F}_2^{n\times n}$) one by one. Therefore, the total space complexity is $O(n^3 + n2^n + n^2)$ bits.

5 Conclusion

We have introduced an algorithm that accelerates the method proposed in [27] by a factor of approximately $\frac{2^n}{n^2}$. Using this algorithm, we performed an exhaustive search in dimension 10 and found that there are only 3 CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$, all of which correspond to previously known APN functions. We also identified a shorter representative $x^{513} + x^{192} + x^{96} + x^{48} + x^{33} + x^{18} + x^{12}$ for $x^3 + \mathrm{Tr}(x^9)$, meaning that any quadratic APN function over $\mathbb{F}_{2^n}$ with $n \leq 10$ and coefficients in $\mathbb{F}_2$ is CCZ-equivalent to a function with at most n terms with nonzero coefficients. When performing a partial search by restricting the values of the matrix to a smaller set, we found at least 6 CCZ-inequivalent quadratic APN functions with coefficients in $\mathbb{F}_2$ over $\mathbb{F}_{2^{11}}$, all of which are likely CCZ-equivalent to known functions. We used the differential and Walsh spectra of the ortho-derivatives to separate the functions into classes that are guaranteed to be CCZ-inequivalent to one another. For $n = 10$, we applied the equivalence algorithm from [9] to verify that the functions within each class are pairwise CCZ-equivalent. For $n = 11$, we found 29 quadratic APN functions with coefficients in $\mathbb{F}_2$, which can be divided into 6 classes according to the differential and Walsh spectra of their ortho-derivatives. Although it is currently not possible to verify that the functions within each of these classes are CCZ-equivalent using existing methods, we can conclude that there are at least 6 CCZ-equivalence classes represented by these functions. Based on the accuracy of the Walsh and differential

spectra of the ortho-derivatives as invariants, we conjecture that there are precisely 6 CCZ-equivalence classes among these functions.

Acknowledgements Yuyin Yu is supported by the National Key R&D Program of China (Grant No. 2021YFB3100200) and the GuangDong Basic and Applied Basic Research Foundation (Grant No. 2021A1515011904).

Funding Open access funding provided by University of Bergen (incl Haukeland University Hospital)

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Beierle, C., Brinkmann, M., Leander, G.: Linearly Self-Equivalent APN Permutations in Small Dimension. IEEE Trans. Inf. Theory **67**(7), 4863–4875 (2021)
2. Beierle, C., Leander, G.: New Instances of Quadratic APN Functions. IEEE Trans. Inf. Theory **68**(1), 670–678 (2022)
3. Beierle, C., Leander, G.: New Instances of Quadratic APN Functions in Small Dimension. Version 2.1, dataset, Zenodo (2021). https://doi.org/10.5281/zenodo.4738942.
4. Beierle, C., Leander, G., Perrin, L.: Trims and Extensions of Quadratic APN Functions. Des. Codes Crypt. **90**, 1009–1036 (2022)
5. Brinkmann, M., Leander, G.: On the Classification of APN Functions Up to Dimension Five. Des. Codes Crypt. **49**(1–3), 273–288 (2008)
6. Browning, K., Dillon, J.F., McQuistan, M.: APN Polynomials and Related Codes. Special Volume of Journal of Combinatorics. Inf. Syst. Sci. **34**, 135–159 (2009)
7. Budaghyan, L., Calderini, M., Carlet, C., Coulter, R.S., Villa, I.: Constructing APN Functions Through Isotopic Shifts. IEEE Trans. Inf. Theory **66**(8), 5299–5309 (2020)
8. Budaghyan, L., Ivkovic, I., Kaleyski, N.: Triplicate Functions. Cryptogr. Commun. **15**, 35–83 (2023)
9. Ivkovic, I., Kaleyski, N.: Deciding and Reconstructing Linear Equivalence of Uniformly Distributed Functions. Cryptology ePrint Archive. (2022)
10. Budaghyan, L., Carlet, C.: Classes of Quadratic APN Trinomials and Hexanomials and Related Structures. IEEE Trans. Inf. Theory **54**(5), 2354–2357 (2008)
11. Budaghyan, L., Carlet, C., Leander, G.: Two Classes of Quadratic APN Binomials Inequivalent to Power Functions. IEEE Trans. Inf. Theory **54**(9), 4218–4229 (2008)
12. Budaghyan, L., Helleseth, T., Kaleyski, N.: A New Family of APN Quadrinomials. IEEE Trans. Inf. Theory **66**(11), 7081–7087 (2020)
13. Canteaut, A., Couvreur, A., Perrin, L.: Recovering or Testing Extended-Affine Equivalence. IEEE Trans. Inf. Theory **68**(9), 6187–6206 (2022)
14. Biryukov, A., De Canniere, C., Braeken, A., Preneel, B.: A Toolbox for Cryptanalysis: Linear and Affine Equivalence Algorithms. InAdvances in Cryptology-EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22 (2003) (pp. 33-50). Springer Berlin Heidelberg
15. Carlet, C.: Boolean Functions for Cryptography and Coding Theory. Cambridge University Press, Cambridge and New York (2020)
16. Carlet, C., Charpin, P., Zinoviev, V.: Codes, Bent Functions and Permutations Suitable for DES-Like Cryptosystems. Des. Codes Crypt. **15**(2), 125–156 (1998)

17. Edel, Y.: Quadratic APN Functions as Subspaces of Alternating Bilinear Forms. Proceedings of the Contact Forum Coding Theory and Cryptography III, Belgium **2009**, 11–24 (2011)
18. Edel, Y., Kyureghyan, G., Pott, A.: A New APN Function Which Is Not Equivalent to a Power Mapping. IEEE Trans. Inf. Theory **52**(2), 744–747 (2006)
19. Edel, Y., Pott, A.: A New Almost Perfect Nonlinear Function Which Is Not Quadratic. Adv. Math. Commun. **3**(1), 59–81 (2009)
20. Kalgin, K., Idrisova, V.: The Classification of Quadratic APN Functions in 7 Variables and Combinatorial Approaches to Search for APN Functions. Cryptogr. Commun. **15**, 239–256 (2023)
21. Gold, R.: Maximal Recursive Sequences with 3-valued Recursive Cross Correlation Functions. IEEE Trans. Inf. Theory **IT–14**(1), 154–156 (1968)
22. Taniguchi, H.: On Some Quadratic APN Functions. Des. Codes Crypt. **87**(9), 1973–1983 (2019)
23. Perrin L.: How to Take a Function Apart with SboxU. The 5th International Workshop on Boolean Functions and their Applications (BFA 2020) (2020)
24. Weng, G., Tan, Y., Gong, G.: On Quadratic Almost Perfect Nonlinear Functions and Their Related Algebraic Object, in Proc. Int. Workshop Coding Cryptogr. (WCC), (2013)
25. Yoshiara, S.: Equivalence of Quadratic APN Functions. J. Algebraic Combin. **35**, 461–475 (2012)
26. Yu, Y., Perrin, L.: Constructing More Quadratic APN Functions with the QAM Method. Cryptogr. Commun. **14**(6), 1359–1369 (2022)
27. Yu, Y., Kaleyski, N., Budaghyan, L., Li, Y.: Classification of Quadratic APN Functions with Coefficients in $\mathbb{F}_2$ for Dimensions up to 9. Finite Fields and Their Applications **68**, 101733 (2020)
28. Yu, Y., Wang, M., Li, Y.: A Matrix Approach for Constructing Quadratic APN Functions. IACR Cryptol. ePrint Arch., Tech. Rep. 2013/007, (2013)
29. Yu, Y., Wang, M., Li, Y.: A Matrix Approach for Constructing Quadratic APN Functions. Des. Codes Crypt. **73**, 587–600 (2014)
30. Lijing, Z., Haibin, K., Yanjun, L., Jie, P., Deng, T.: Constructing New APN Functions Through Relative Trace Functions. IEEE Trans. Inf. Theory **68**(11), 7528–7537 (2022)
31. Cannon, W., Bosma, J.a., Fieker, C., Steel, A.: Handbook Magma Functions, 2nd ed. (2014) [Online]. Available: https://www.math.uzh.ch/sepp/magma-2.20.4-cr/

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.